6η Εργαστηριακή Άσκηση

Μάθημα «Εισαγωγή στη Σχεδίαση Συστημάτων VLSI»

Τμήμα 2, Ομάδα 1

Κωνσταντίνος Τριαντάφυλλος, ΑΜ: 03122906

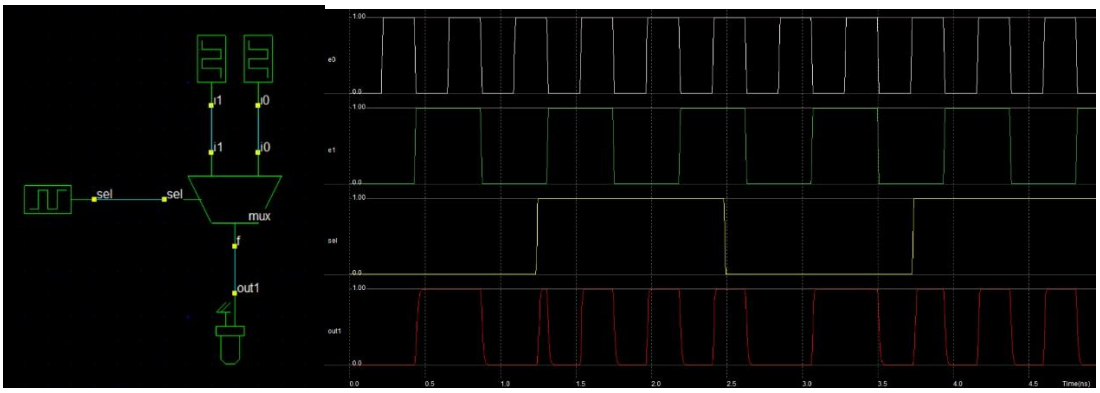
Γριζανίτης Ευθύμιος, ΑΜ: 03122876

A2.

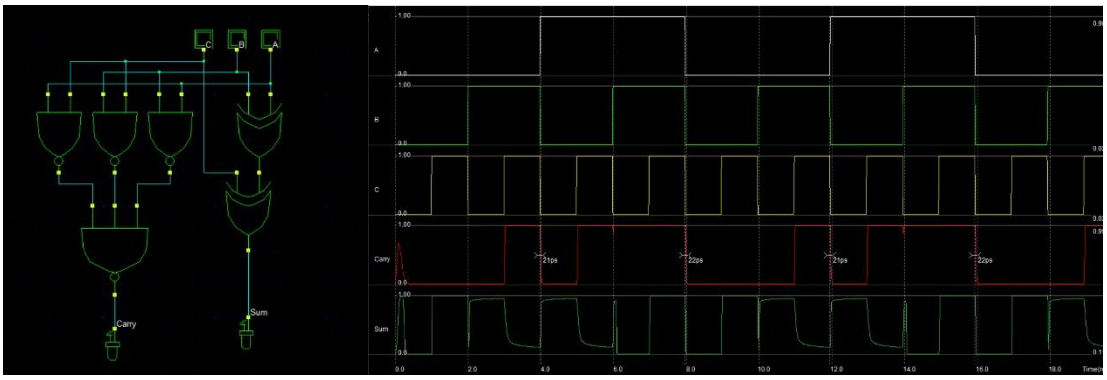
Για την υλοποίηση της 3-bit ALU επιλέξαμε να χρησιμοποιήσουμε τον 1ο τρόπο σχεδίασης, δηλαδή να σχεδιάσουμε αρχικά το κύκλωμα στο πρόγραμμα DSCH, και έτσι να παραχθεί αυτόματα ο κώδικας Verilog HDL, τον οποίο βάλαμε στο microwind και παράχθηκε το layout.

Χρησιμοποιήσαμε τις λογικές πύλες που μας παρέχει το DSCH (AND, NOT, XOR, AND, NAND).

Παρακάτω φαίνεται ο mux που χρησιμοποιήθηκε:

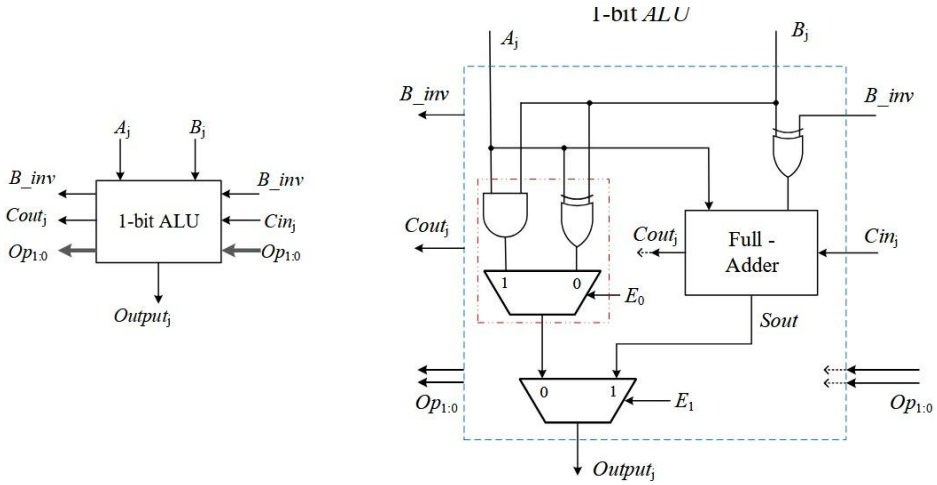


Και ο full-adder:

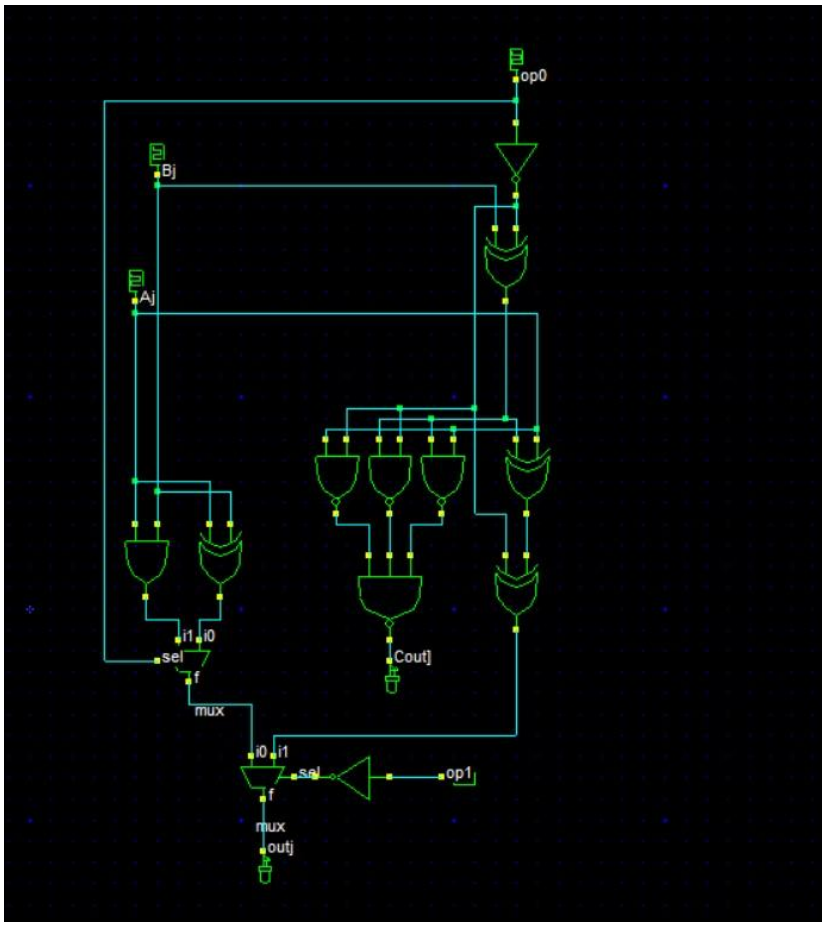


Και ελέγχθηκε η σωστή λειτουργία τους.

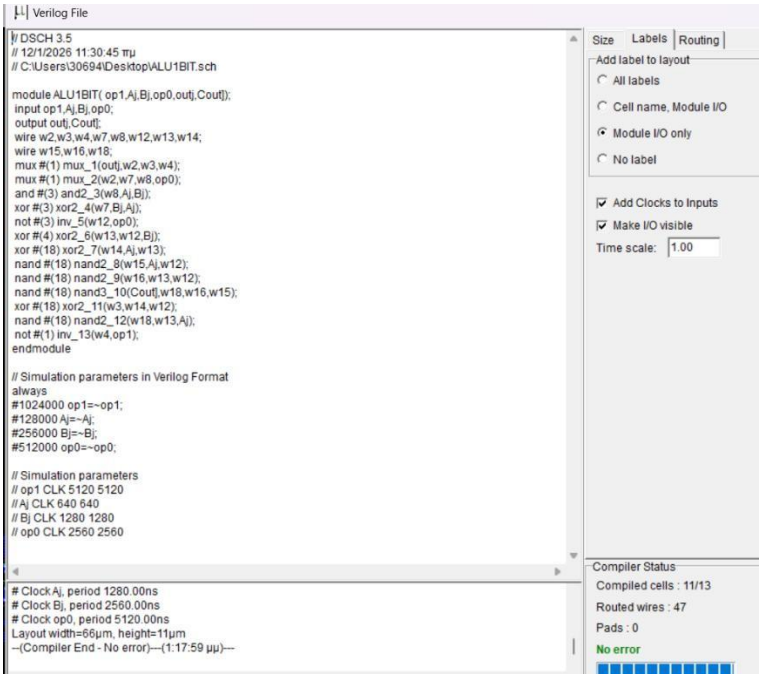
Έπειτα προχωρήσαμε στη σχεδίαση της 1-bit ALU ακολουθώντας τα παρακάτω σχήματα:

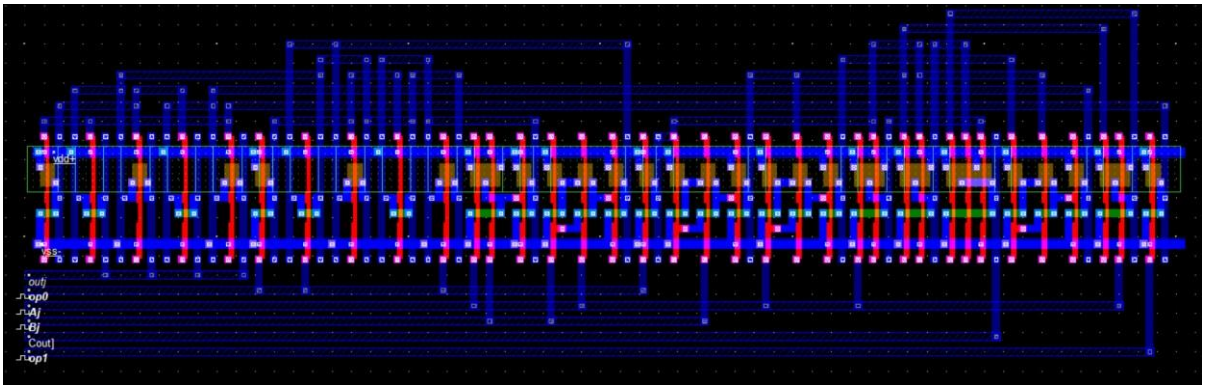


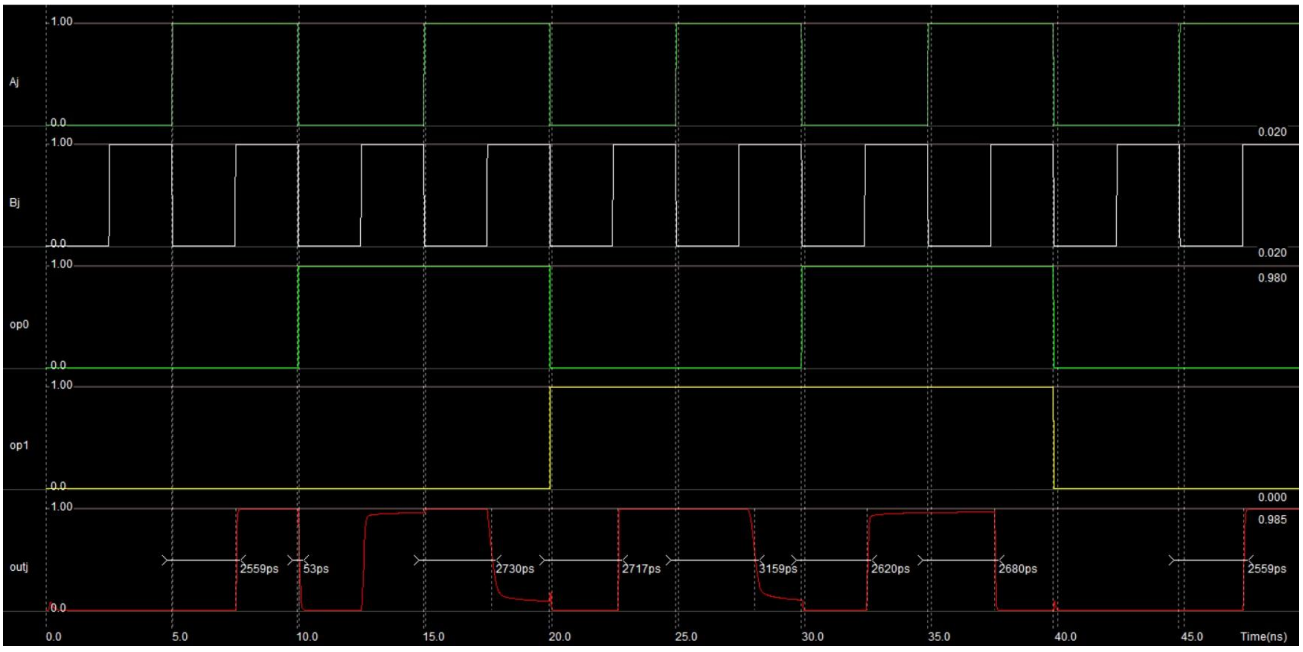
1-bit ALU σχεδιασμένη στο DSCH:



Κάναμε File→Make Verilog File για να ελέγξουμε την ορθότητά της στο microwind:

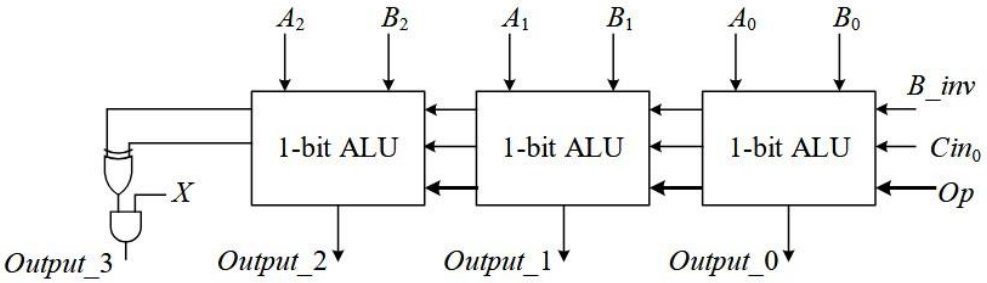






(Με τη σειρά AND, XOR. Αφαίρεση, Πρόσθεση, όπως δηλώνει ο κωδικός Op[1:0])

Τέλος, φτιάξαμε την 3-bit ALU ακολουθώντας το σχήμα:



Σχήμα 6.2: 3-bit ALU

Δηλαδή, χρησιμοποιήσαμε 3 1-bit ALUs που φτιάξαμε πριν και:

•Το Cout0 συνδέθηκε σαν Cin1, το Cout1 σαν Cin2

•Το Cout 2 μαζί με το B_inv περάστηκαν σε μια XOR

•Η έξοδος αυτής της XOR και το X περάστηκαν σε μια AND, η έξοδος της οποίας είναι το Output_3

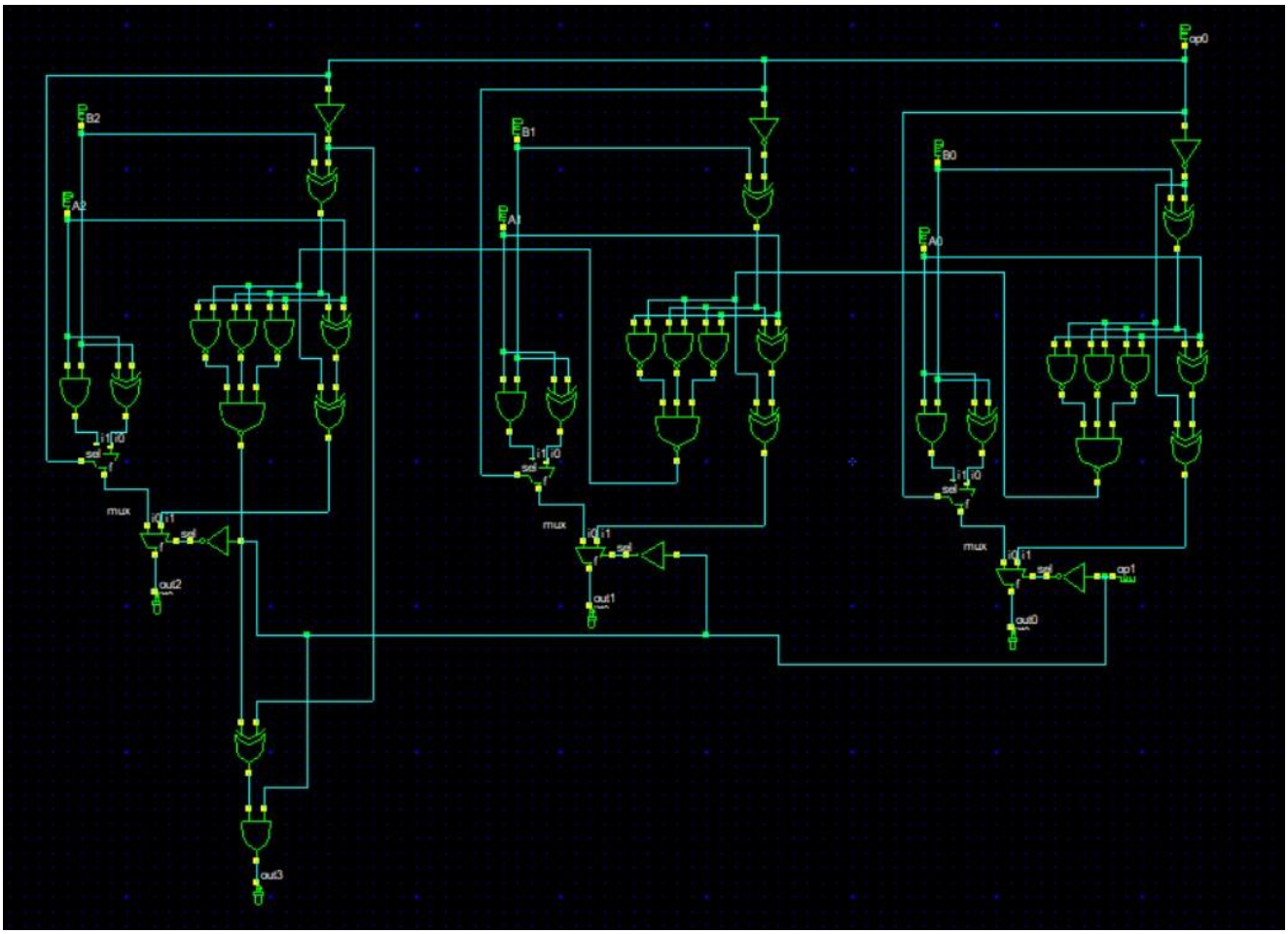
•Αφού κατασκευάσαμε τον παρακάτω πίνακα με βάση την εκφώνηση, συνειδητοποιήσαμε πως X=op_1 και B_inv=Cin0=op_0'

	op_1	op_0	B_inv	Cin	X
AND	0	0			0
XOR	0	1			0
ΑΦΑΙΡΕΣΗ	1	0	1	1	1
ΠΡΟΣΘΕΣΗ	1	1	0	0	1

(Το X πήρε τις συγκεκριμένες τιμές, ώστε στις λογικές πράξεις να μηδενίζει το output_3 (εφόσον είναι μαζί στην πύλη AND).

Στον δεύτερο mux, τοποθετήθηκε μια πύλη NOT στην είσοδο ελέγχου πριν το op1 ώστε να λειτουργεί σωστά, διότι και οι είσοδοί του ήταν αντεστραμμένες(i0,i1)

Έτσι, το κύκλωμα που σχεδιάσαμε στο DSCH για την ALU-3bit είναι το εξής:



op_1=0:

Ο 1ος mux επιλέγει εάν θα γίνει η πράξη AND ή XOR. Αυτό εξαρτάται από το op0, οπότε το op0 μπαίνει ως είσοδος ελέγχου σε αυτό τον mux. **op_1=1:**

Αφού B_inv=op0', →όταν op0=1(πρόσθεση), b_inv=0=Cin, άρα περνά Bj και γίνεται A+B

→και όταν op0=0(αφαίρεση), b_inv=1=Cin, άρα περνά Bj' και γίνεται A+B'+1

Ο 2ος mux επιλέγει αν θα περάσει μία από τις XOR/AND, δηλαδή πράξη του πρώτου mux ή μία από τις πρόσθεση/αφαίρεση, δηλαδή πράξη του full-adder, αφού ως είσοδος ελέγχου του είναι το op1', και όταν op1 = 1 έχουμε πρόσθεση/αφαίρεση και όταν op1 = 0 έχουμε XOR/AND.

Ο κώδικας Verilog της 3bit-ALU:

Verilog File

```
// DSCH 3.5
// 12/1/2026 11:46:23 πμ
// C:\Users\30694\Desktop\ALU3BIT.sch

module ALU3BIT( A2,B2,op1,A0,B0,op0,A1,B1,
 out2,out0,out1,out3);
 input A2,B2,op1,A0,B0,op0,A1,B1;
 output out2,out0,out1,out3;
 wire w2,w3,w4,w6,w7,w9,w10,w11;
 wire w14,w15,w16,w18,w19,w20,w21,w22;
 wire w25,w27,w28,w29,w31,w32,w35,w36;
 wire w37,w38,w39,w40,w41,w42,w43,w44;
 wire w45,w46,w47,w48,;
 mux #(1) mux_1(out2,w2,w3,w4);
 mux #(1) mux_2(w9,w6,w7,op0);
 mux #(1) mux_3(w2,w10,w11,op0);
 and #(3) and2_4(w11,A2,B2);
 xor #(3) xor2_5(w10,B2,A2);
 not #(2) inv_6(w14,op0);
 mux #(1) mux_7(out1,w9,w15,w16);
 xor #(4) xor2_8(w18,w14,B2);
 xor #(18) xor2_9(w19,A2,w18);
 nand #(18) nand2_10(w21,A2,w20);
 nand #(18) nand2_11(w22,w18,w20);
 xor #(3) xor2_12(w6,B1,A1);
 nand #(20) nand3_13(op1,w25,w22,w21);
 xor #(18) xor2_14(w3,w19,w20);
 nand #(18) nand2_15(w25,w18,A2);
 not #(1) inv_16(w4,op1);
 mux #(1) mux_17(out0,w27,w28,w29);
 mux #(1) mux_18(w27,w31,w32,op0);
 and #(3) and2_19(w32,A0,B0);
 xor #(3) xor2_20(w31,B0,A0);
 not #(3) inv_21(w35,op0);
 xor #(4) xor2_22(w36,w35,B0);
 xor #(18) xor2_23(w37,A0,w36);
 nand #(18) nand2_24(w38,A0,w35);
```

```
 xor #(18) xor2_23(w37,A0,w36);
 nand #(18) nand2_24(w38,A0,w35);
 nand #(18) nand2_25(w39,w36,w35);
 nand #(19) nand3_26(w41,w40,w39,w38);
 xor #(18) xor2_27(w28,w37,w35);
 nand #(18) nand2_28(w40,w36,A0);
 not #(1) inv_29(w29,op1);
 not #(1) inv_30(w16,op1);
 nand #(18) nand2_31(w43,w42,A1);
 xor #(18) xor2_32(w15,w44,w41);
 nand #(19) nand3_33(w20,w43,w45,w46);
 nand #(18) nand2_34(w45,w42,w41);
 nand #(18) nand2_35(w46,A1,w41);
 xor #(18) xor2_36(w44,A1,w42);
 xor #(4) xor2_37(w42,w47,B1);
 and #(3) and2_38(w7,A1,B1);
 not #(1) inv_39(w47,op0);
 xor #(3) xor2_40(w48,w14,op1);
 and #(3) and2_41(out3,w48,op1);
endmodule

// Simulation parameters in Verilog Format
always
#128000 A2=~A2;
#256000 B2=~B2;
#1024000 op1=~op1;
#128000 A0=~A0;
#256000 B0=~B0;
#512000 op0=~op0;
#128000 A1=~A1;
#256000 B1=~B1;

// Simulation parameters
// A2 CLK 640 640
// B2 CLK 1280 1280
// op1 CLK 5120 5120
// A0 CLK 640 640
```

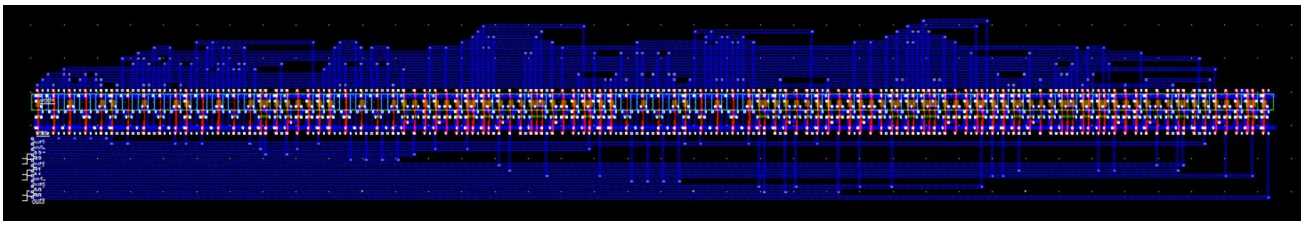
```
#128000 A1=~A1;
#256000 B1=~B1;

// Simulation parameters
// A2 CLK 640 640
// B2 CLK 1280 1280
// op1 CLK 5120 5120
// A0 CLK 640 640
// B0 CLK 1280 1280
// op0 CLK 2560 2560
// A1 CLK 640 640
// B1 CLK 1280 1280
```

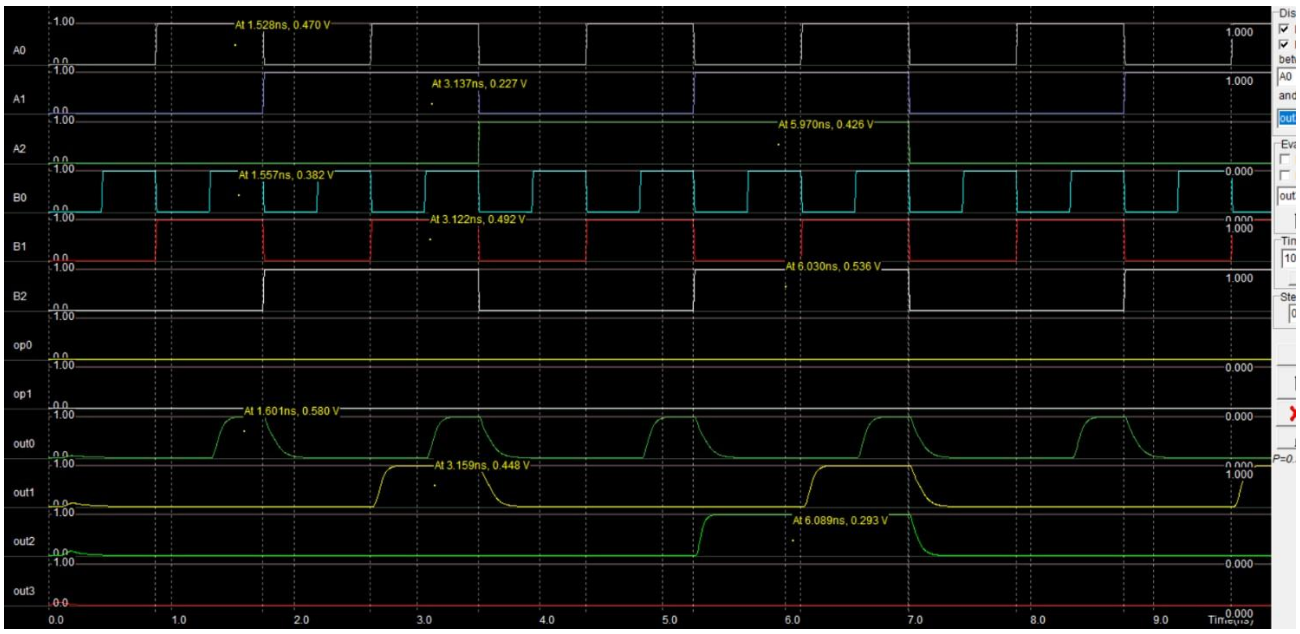
```
# Clock op0, period 5120.00ns
# Clock A1, period 1280.00ns
# Clock B1, period 2560.00ns
Layout width=66μm, height=10μm
--(Compiler End - No error)---(1:14:18 μμ)---
```

Compiler Status
Compiled cells : 35/41
Routed wires : 144
Pads : 0
No error

Και το layout που προέκυψε στο microwind:



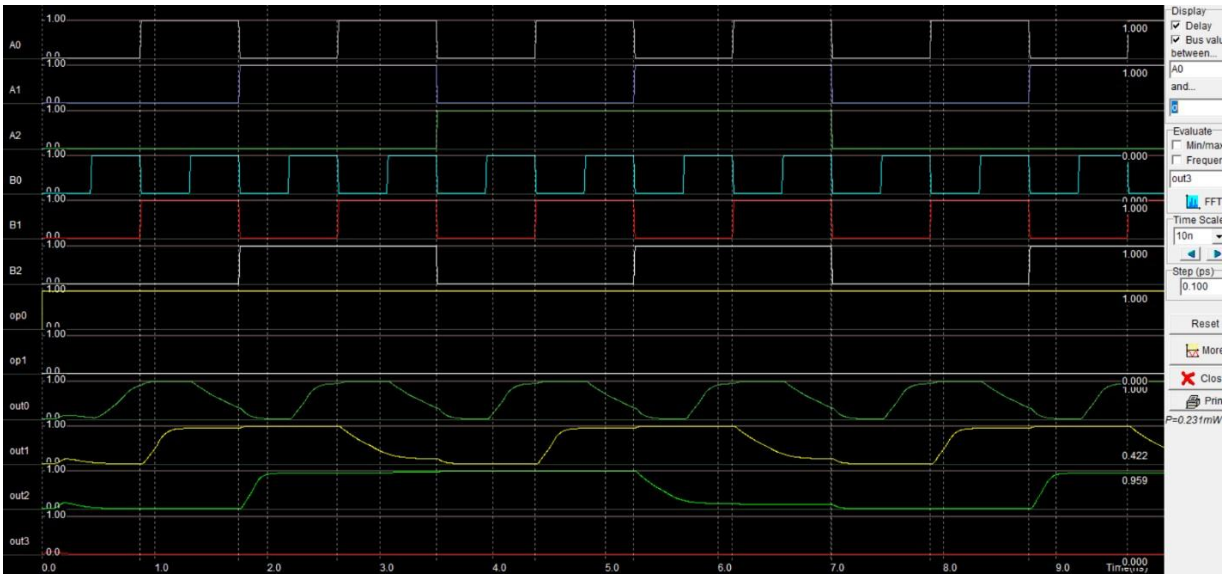
Παρακάτω φαίνονται οι προσομοιώσεις όλων των πράξεων: **AND→00**



Βλέπουμε σωστά αποτελέσματα στα out0, out1, out2 (=1 όταν τα Aj,Bj είναι ταυτόχρονα 1), και out3=0 όπως θα θέλαμε για τις λογικές πράξεις.

Aj	**Bj**	**Outputj**
0	0	0
0	1	0
1	0	0
1	1	1

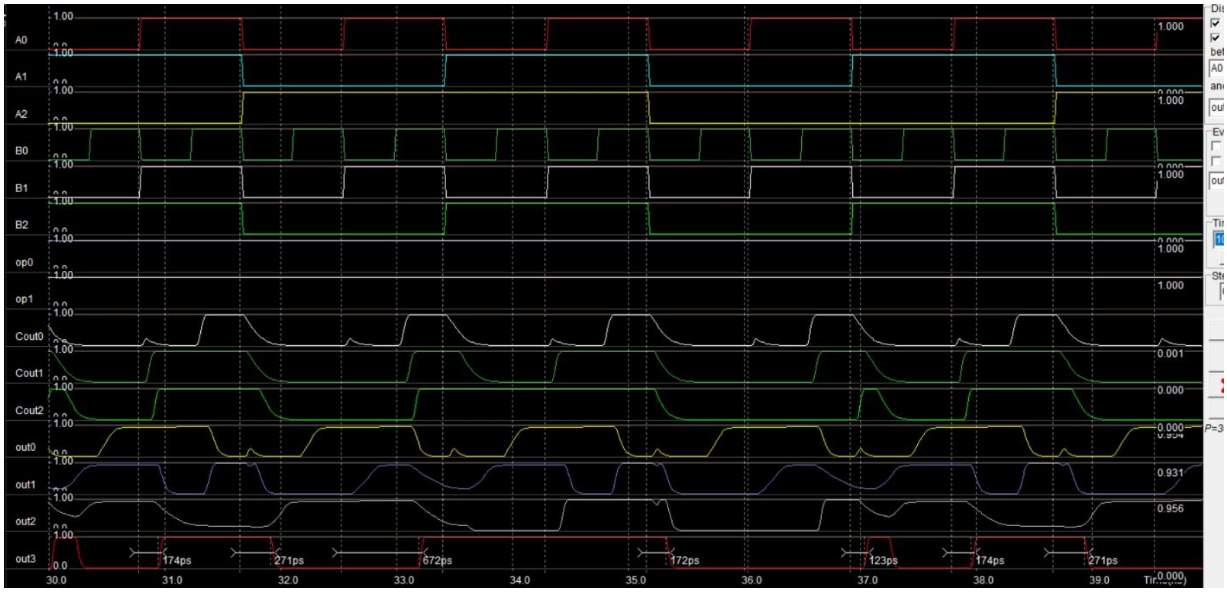
XOR→01



Βλέπουμε σωστές τιμές στα out0,out1,out2 και το out3=0 και σε αυτή τη λογική πράξη.

Aj	**Bj**	**Outputj**
0	0	0
0	1	1
1	0	1
1	1	0

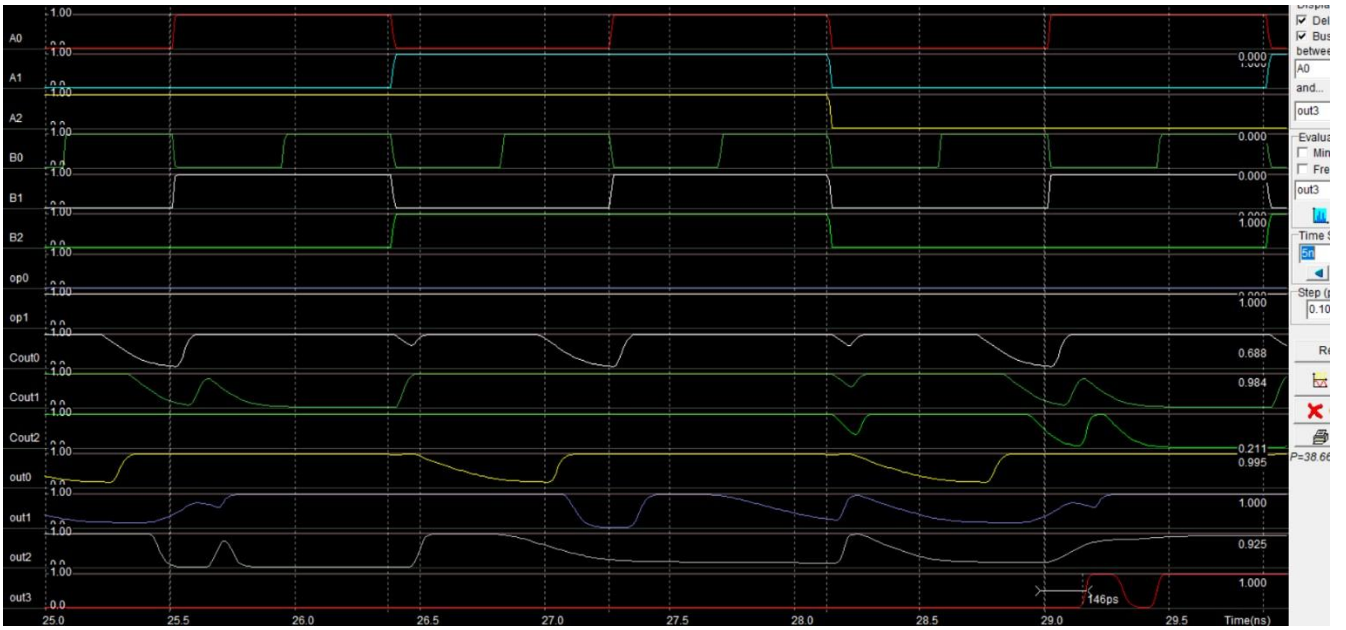
Πρόσθεση →11



Προέκυψαν οι επιθυμητές τιμές στα out0,out1,out2,cout0,cout1,cout2 κι επίσης παρατηρούμε ότι το out3 έχει τη συμπεριφορά του cout2, το οποίο είναι λογικό διότι στη πρόσθεση πάντα op1=1(=X) και αφού op1, cout2 είναι στη πύλη AND, out3=1 όταν cout2=1.

Aj	**Bj**	**Coutj**	**Outputj**
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Αφαίρεση →10

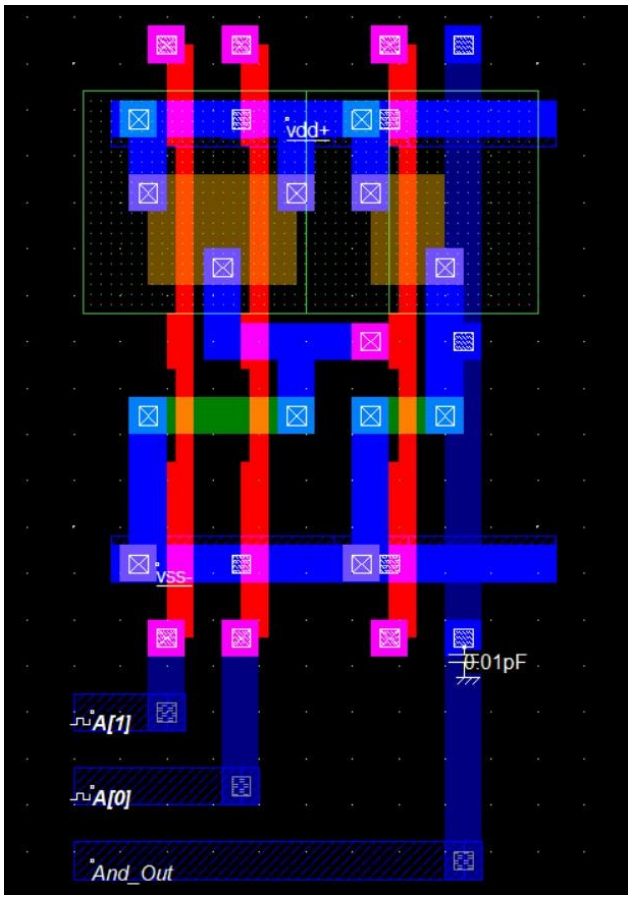


Τα αποτελέσματα είναι επιθυμητά (A+B'+1), και βλέπουμε ότι εδώ το out3 έχει συμπεριφορά αντίστροφη από το cout2(για να είναι σωστό το αποτέλεσμα χρειάζεται αντιστροφέας μετά το Cout2).

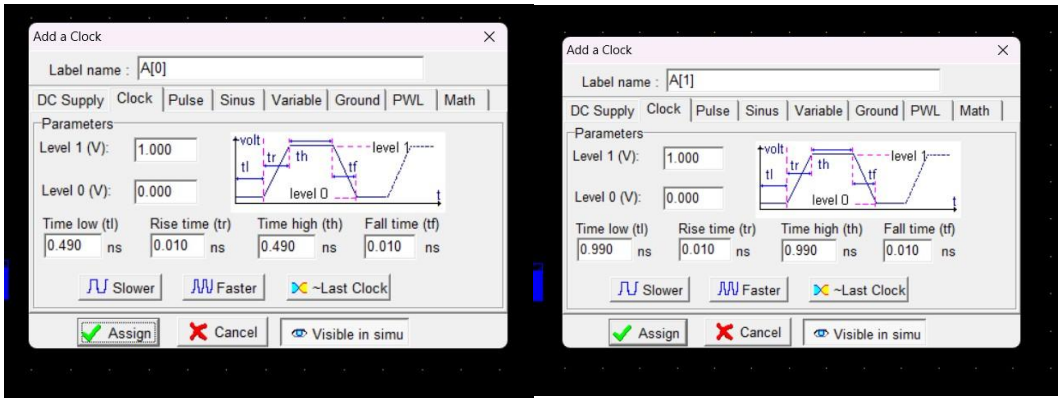
Aj	**Bj**	**Coutj**	**Outputj**
0	0	1	0
0	1	0	1
1	0	1	1
1	1	1	0

B2.

1.Κάνοντας compile → Compile Verilog File → And.v → Compile → Back to editor, προέκυψε το παρακάτω layout στην έξοδο του οποίου προσθέσαμε τον χωρητικό φορτίο 0.01pF.

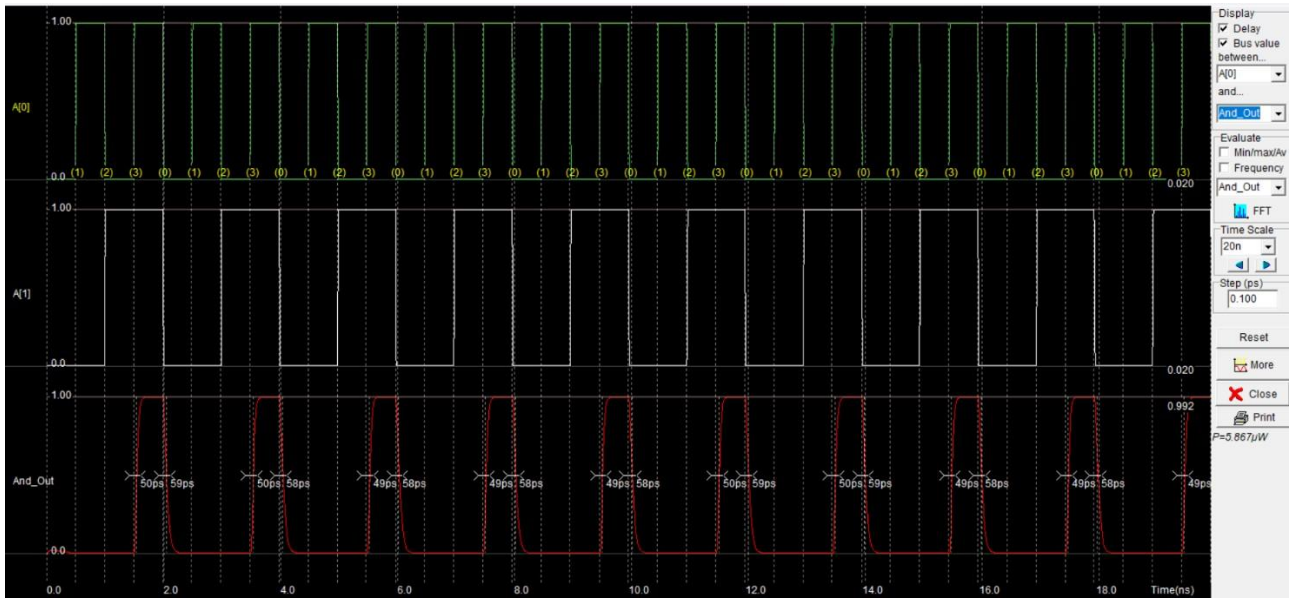


a.Για να προκύψει A[0]=1nsec, A[1]=2nsec βάλαμε τις παρακάτω τιμές στα tl,tr,th,tf (αφού T=tl+tr+th+tf).



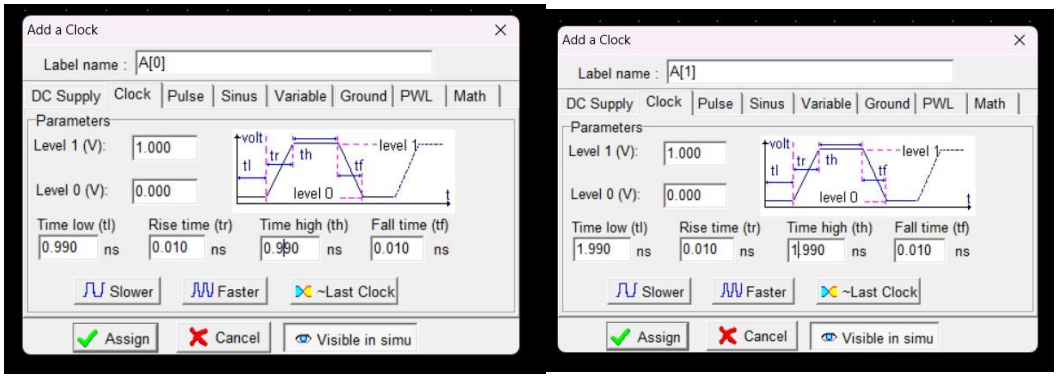
Τρέξαμε την προσομοίωση σε Time Scale = 20nsec και πήραμε τη δυναμική ισχύ που καταναλώθηκε από κάτω δεξιά.

Ξέρουμε πως Ptotal = Pdynamic + Pstatic, όμως αφού στο πρόγραμμα microwind η στατική ισχύς δεν υπολογίζεται, εμείς παίρνουμε τη δυναμική ισχύ.

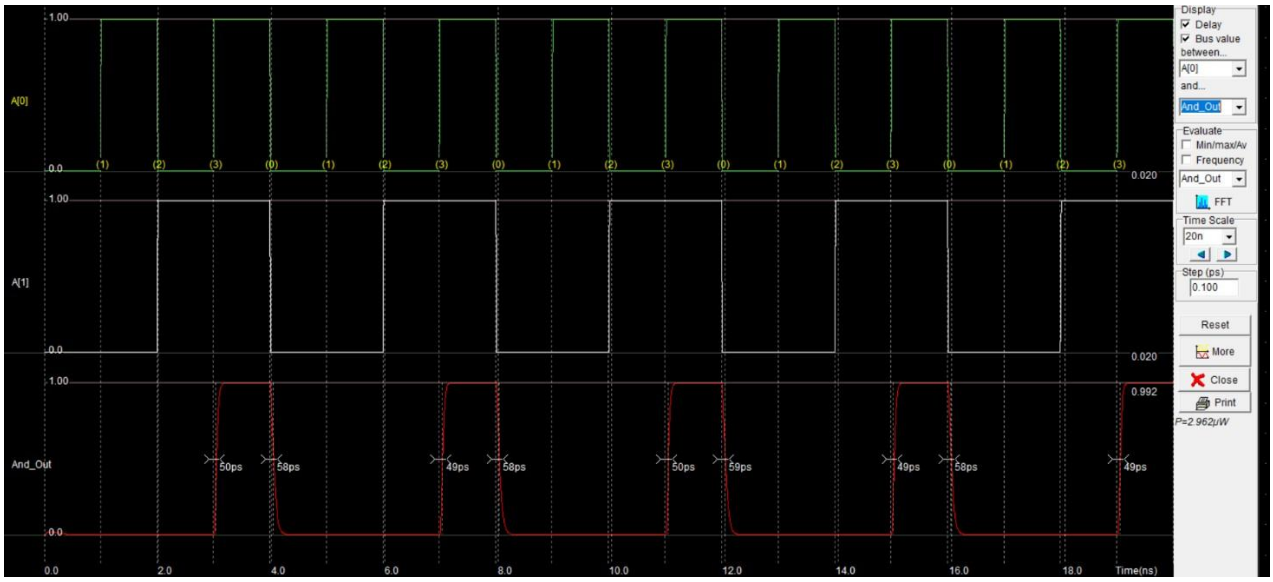


P=5.867µW

b.Για να υποδιπλασιαστεί η συχνότητα(δηλαδή να διπλασιαστεί η περίοδος), πατήσαμε Slower στα A[0],A[1].

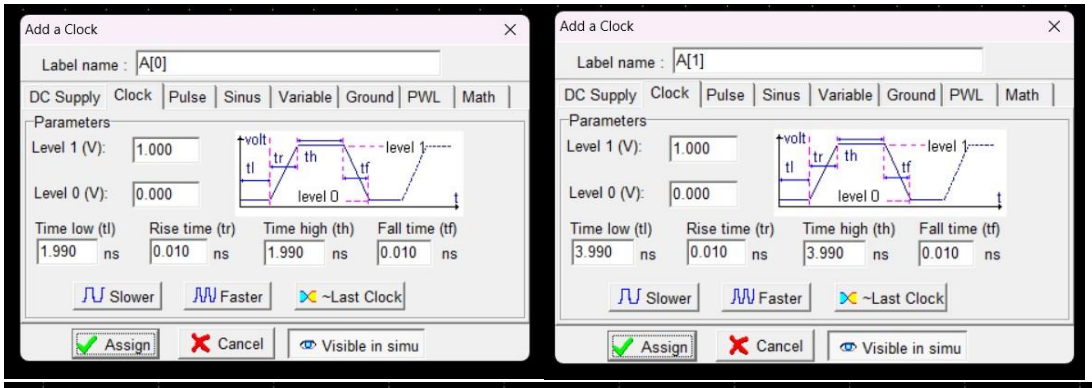


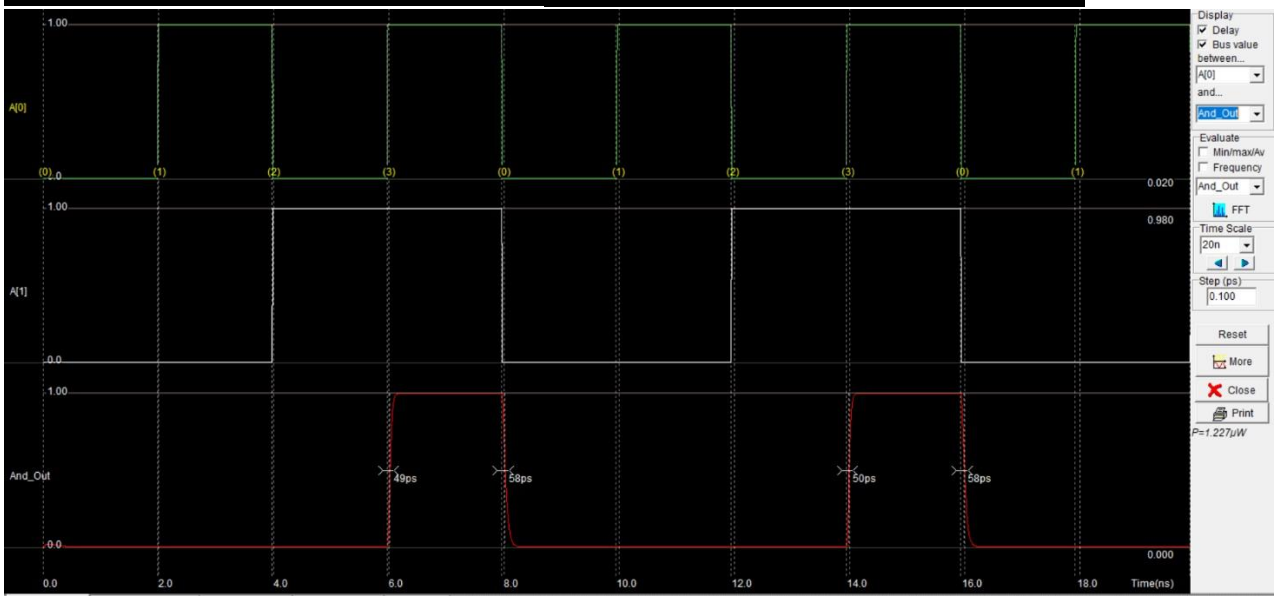
Τρέξαμε ξανά τη προσομοίωση:



Εδώ λάβαμε P=2.962μW

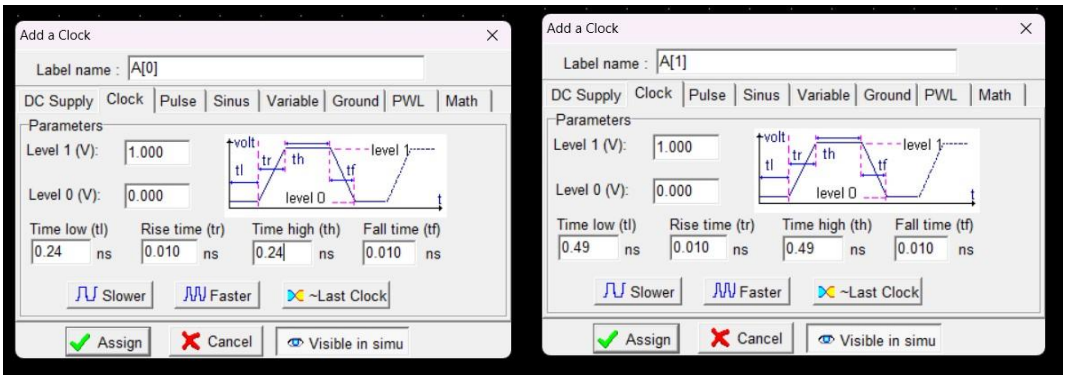
c. Ομοίως, υποδιπλασιάσαμε ξανά τη συχνότητα με Slower:



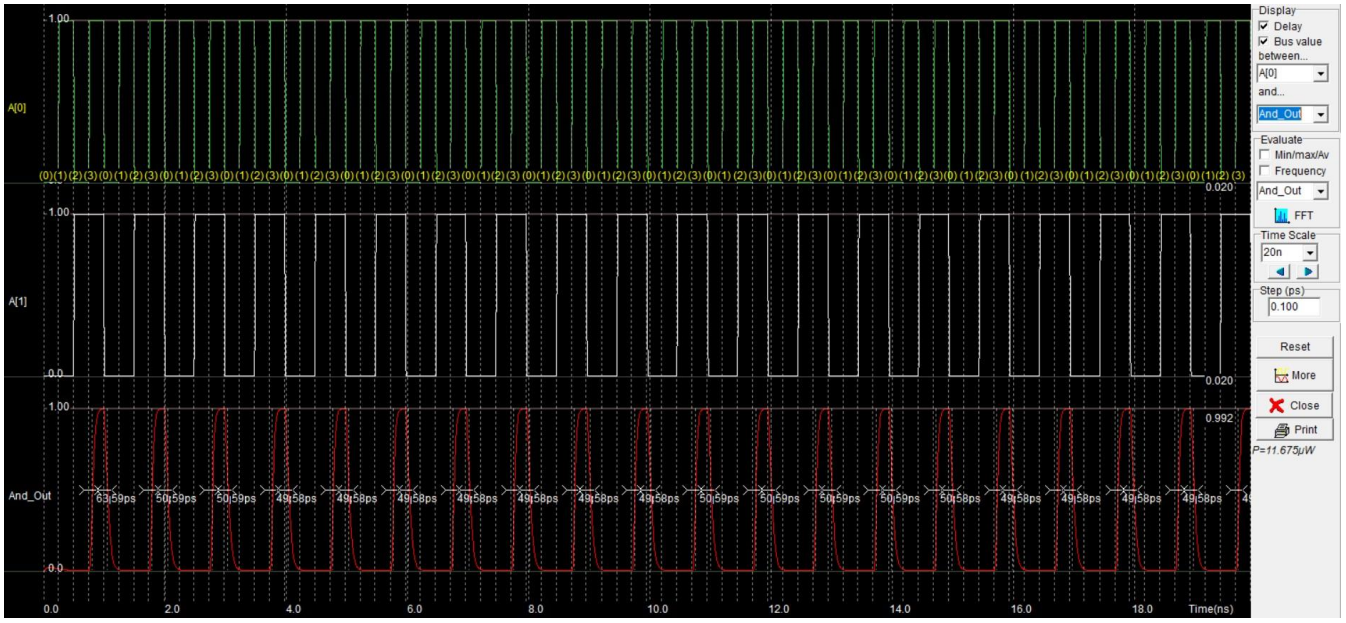


P=1.227μW

d.Για να προκύψει A[0]=0.5nsec, A[1]=1nsec βάλαμε τις παρακάτω τιμές στα tl,tr,th,tf (αφού T=tl+tr+th+tf).



Τρέξαμε ξανά τη προσομοίωση:



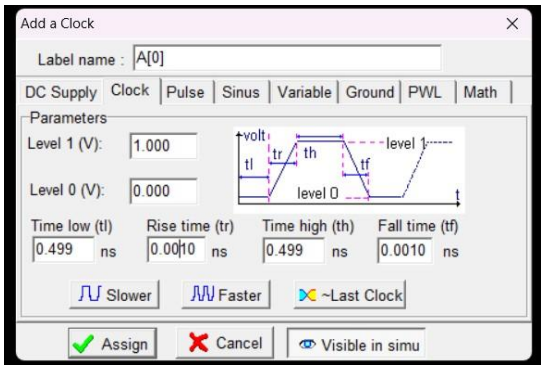
Εδώ έχουμε P=11.675μW

Αυτό που παρατηρήσαμε από τις μετρήσεις μας είναι ότι όσο πιο μεγάλη είναι η συχνότητα, δηλαδή όσο πιο μικρή η περίοδος, τόσο μεγαλύτερη είναι η δυναμική ισχύς που καταναλώνεται στην πύλη And. Αυτό επιβεβαιώνεται από τον τύπο

Pdynamic= Pswitching + Pshort-circuit, αφού Pswitching = α·C· VDD2·f , ο οποίος υποδηλώνει πως η δυναμική ισχύς είναι ανάλογη της συχνότητας.

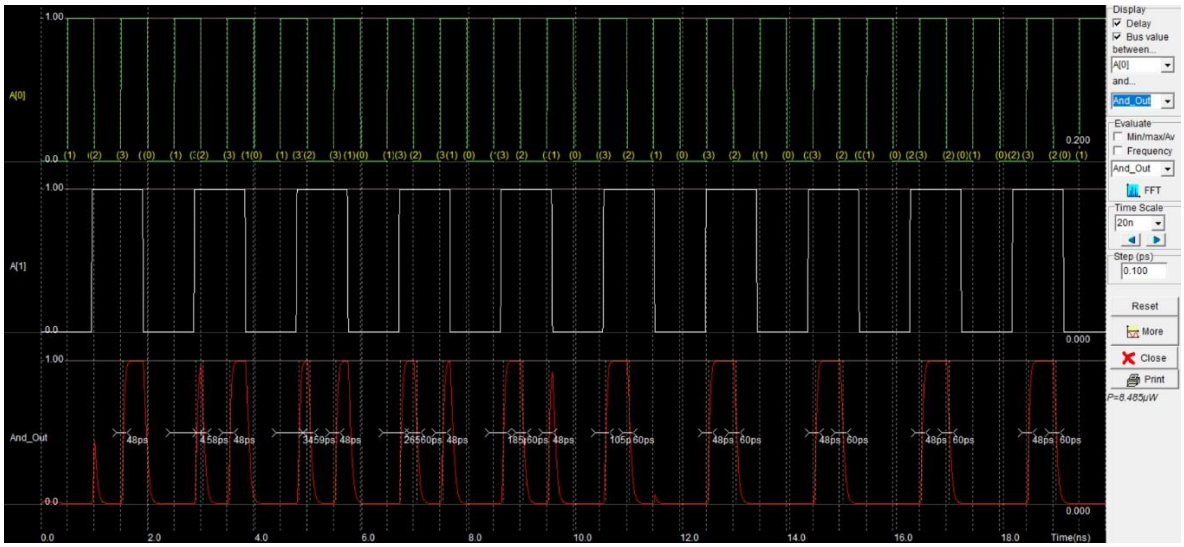
2.

Για να είναι η περίοδος του A[0] 1 nsec, ορίσαμε τις εξής τιμές:



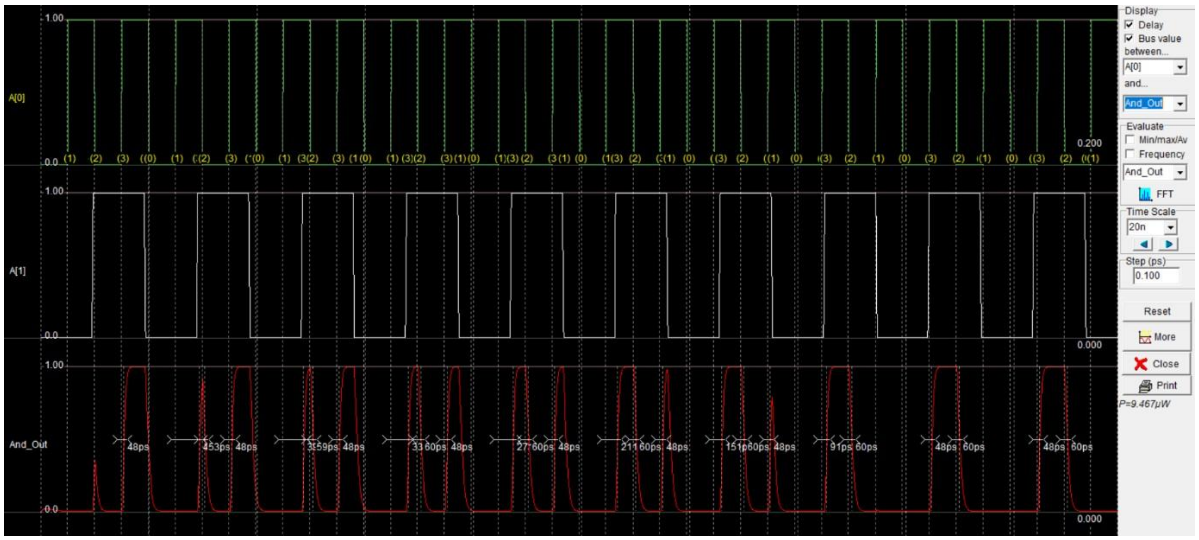
Στο A[1] ορίσαμε tlow=0.950nsec και thigh=0.950 και δοκιμάσαμε 5 τιμές για τα rise time (tr) και fall time (tf), καταγράφοντας κάθε φορά την ισχύ που καταναλώνεται, όπως στην προηγούμενη άσκηση.

a) tr = tf = 0.01:



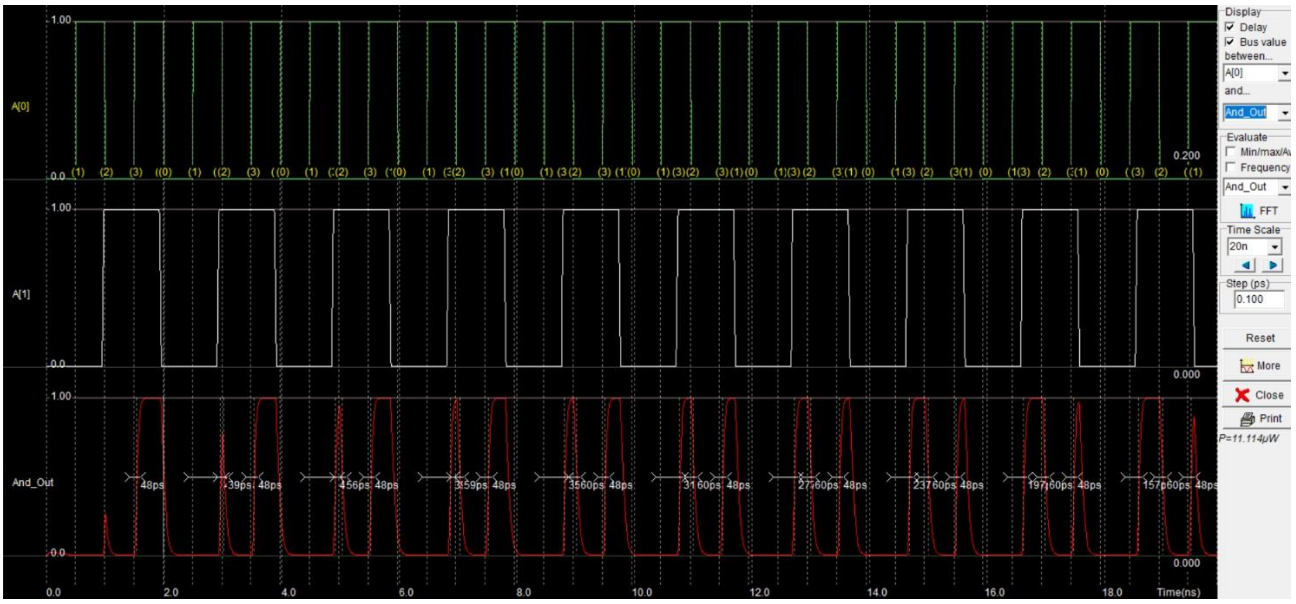
P=8.485µW

b) tr = tf = 0.02:



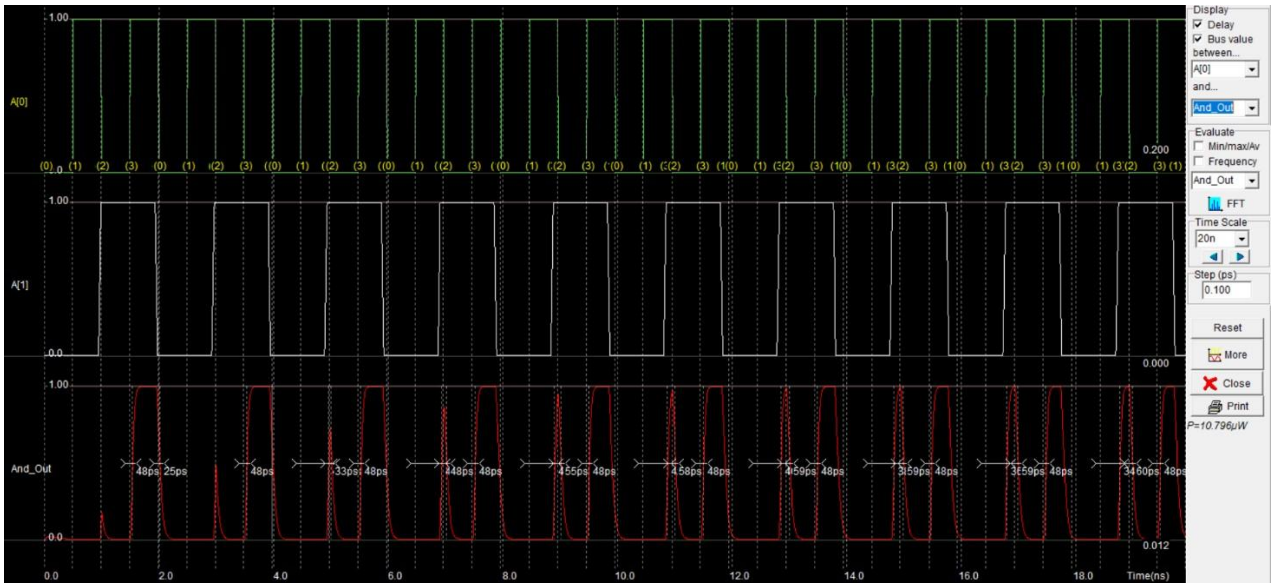
P=9.467µW

c) tr = tf = 0.03:



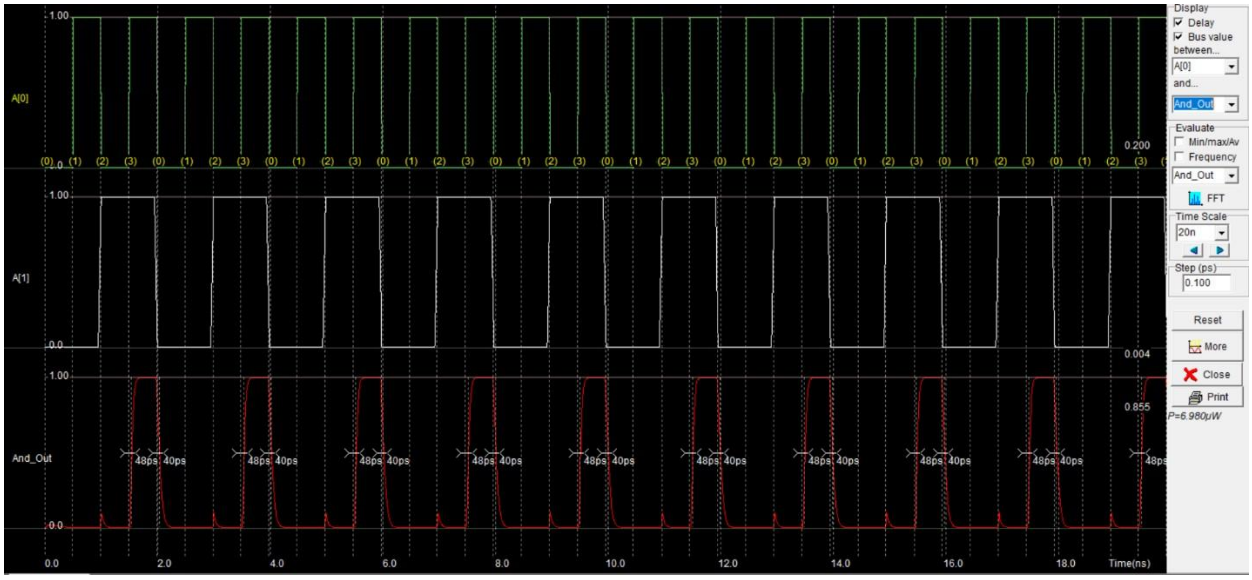
P=11.114µW

d) tr = tf = 0.04:



P=10.796µW

e) tr = tf = 0.05:

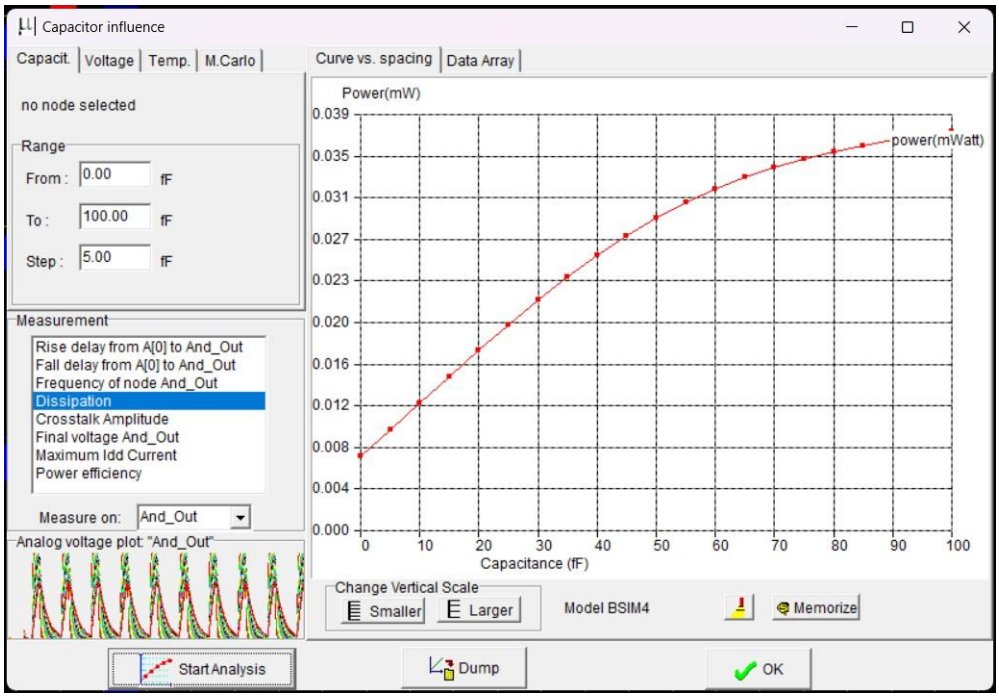


P=6.980µW

Δε παρατηρείται σταθερή συμπεριφορά της ισχύος, αφού αρχικά αυξάνεται με την αύξηση συχνότητας και μετά μειώνεται. Στις περιπτώσεις που αυξάνεται η ισχύς βλέπουμε επίσης ότι το σήμα εξόδου δεν είναι πολύ καθαρό, διότι δεν επιτυγχάνεται πάντα ο ιδανικός συγχρονισμός μεταξύ των 2 ρολογιών.

Το πιο καθαρό σήμα και η πιο μικρή ισχύς παρατηρείται για tr=tf=0.05.

3.Αφού κάναμε Analysis → Parametric Analysis → επιλέξαμε Capacitance → Dissipation → Start Analysis, προέκυψε η εξής γραφική παράσταση:

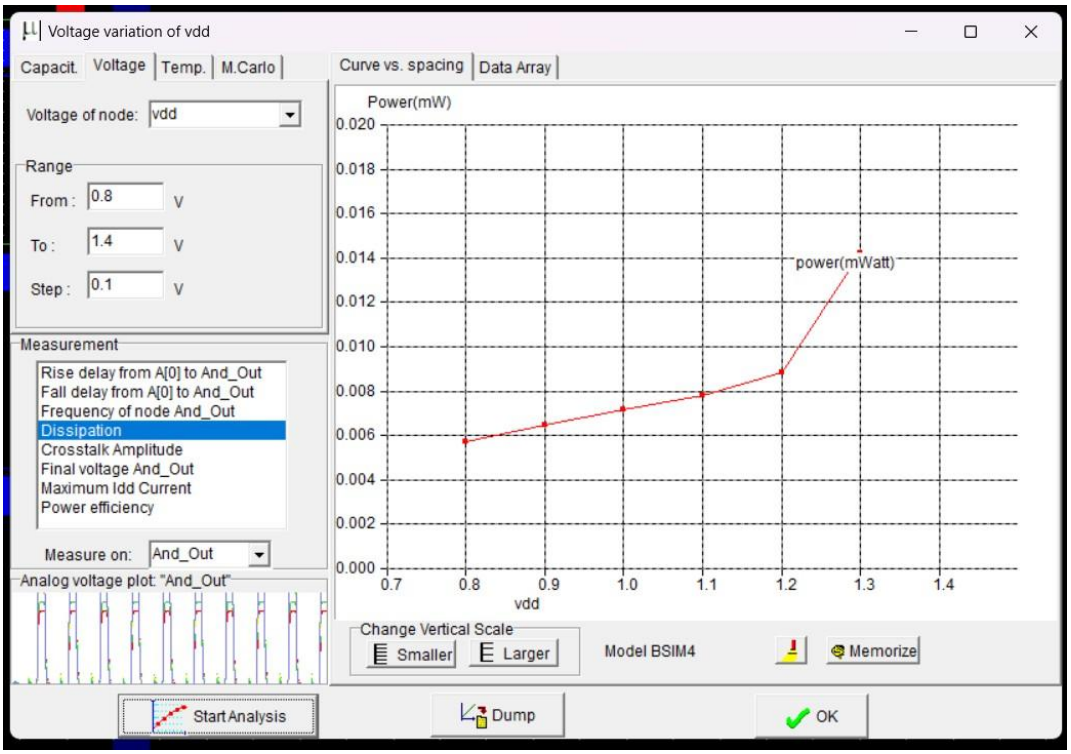


Από τη γραφική παράσταση παρατηρούμε ότι η ισχύς αυξάνεται γραμμικά με την αύξηση της χωρητικότητας του πυκνωτή. Αυτό επιβεβαιώνεται και από τον τύπο Pswitching = α·C· VDD^2·f (αφού Pdynamic= Pswitching + Pshort-circuit, και εμείς αναφερόμαστε μόνο στην Pdynamic στο πρόγραμμα microwind).

Οι ακριβείς τιμές που μας έδωσε το Dump:

```
Capa (fF)   power(mWatt)
0.000 0.007
5.000 0.009
10.000 0.012
15.000 0.014
20.000 0.017
25.000 0.019
30.000 0.022
35.000 0.024
40.000 0.026
45.000 0.028
50.000 0.029
55.000 0.031
60.000 0.032
65.000 0.033
70.000 0.034
75.000 0.035
80.000 0.036
85.000 0.036
90.000 0.037
95.000 0.037
100.000 0.038
```

4. Ακολουθώντας τα ίδια βήματα με πριν, όμως επιλέγοντας Voltage και τις κατάλληλες τιμές, προέκυψε η παρακάτω γραφική παράσταση:



Η ισχύς αυξάνεται σχεδόν τετραγωνικά σε σχέση με τη Vdd. Αυτό και πάλι συμφωνεί με τον τύπο Pswitching = α·C· VDD^2·f . Οι ακριβείς τιμές που μας έδωσε το Dump:

```
vdd(V)   power(mWatt)
0.800 0.006
0.900 0.006
1.000 0.007
1.100 0.008
1.200 0.009
1.300 0.014
```